



FitLife Gym

Gestión de Bases de Datos

Proyecto Intermodular 1º ASIR

1. Análisis de los datos del sistema

FitLife Gym es un gimnasio moderno que necesita gestionar de forma eficiente toda la información relativa a sus socios, instalaciones, clases, empleados y pagos. A continuación se identifican las entidades principales y la justificación de por qué deben almacenarse.

1.1 Información que necesita almacenar el sistema

Entidad	Información que almacena	Por qué es necesaria
Socios	Nombre, apellidos, DNI, email, teléfono, fecha de alta, tipo de membresía	Es la entidad central del negocio. Sin ella no se puede gestionar ningún servicio.
Empleados	Nombre, puesto, especialidad, horario, salario	Permite gestionar el personal y asignar instructores a las clases.
Clases	Nombre, descripción, capacidad máxima, sala, horario, instructor	FitLife ofrece clases grupales; hay que controlar aforo y disponibilidad.
Reservas	Socio, clase, fecha, estado (confirmada/cancelada)	Controla quién está inscrito en cada clase y evita sobrepasar el aforo.
Pagos	Socio, importe, fecha, método de pago, concepto	Registro contable imprescindible para gestionar membresías y servicios extra.
Salas	Nombre, capacidad, equipamiento	Permite asignar clases a espacios adecuados según necesidades.
Membresías	Tipo (básica/premium/anual), precio, accesos incluidos	Define las tarifas y los servicios a los que tiene derecho cada socio.

1.2 Relación entre entidades y el funcionamiento del sistema

Todas las entidades están interconectadas para reflejar el funcionamiento real del gimnasio:

- Un socio tiene asignada una membresía que determina sus accesos y precio mensual.
- Un socio puede realizar múltiples reservas en distintas clases.
- Cada clase es impartida por un empleado (instructor) y se desarrolla en una sala concreta.
- Cada reserva genera o está asociada a un pago en la tabla de pagos.
- Las salas tienen capacidad máxima que limita el número de plazas por clase.

2. Diseño de la base de datos

2.1 Modelo Entidad-Relación

A continuación se describe el modelo E/R de FitLife Gym. El diagrama completo en formato editable se encuentra en el repositorio de GitHub (archivo `diagrama_er_fitlife.drawio`).

 El diagrama E/R en draw.io se entrega como archivo separado: `diagrama_er_fitlife.drawio`

2.2 Entidades y atributos

SOCIOS (id_socio PK, nombre, apellidos, dni, email, telefono, fecha_alta, id_membresia FK)

EMPLEADOS (id_empleado PK, nombre, apellidos, puesto, especialidad, email, telefono, salario)

SALAS (id_sala PK, nombre, capacidad, equipamiento)

MEMBRESIAS (id_membresia PK, tipo, precio_mensual, descripcion)

CLASES (id_clase PK, nombre, descripcion, capacidad_max, horario_inicio, horario_fin, dia_semana, id_sala FK, id_empleado FK)

RESERVAS (id_reserva PK, id_socio FK, id_clase FK, fecha_reserva, estado)

PAGOS (id_pago PK, id_socio FK, importe, fecha_pago, metodo_pago, concepto)

2.3 Relaciones entre tablas

- SOCIOS (N) ↔ (1) MEMBRESIAS — Un socio tiene una membresía; una membresía puede tener muchos socios.
- SOCIOS (1) ↔ (N) RESERVAS — Un socio puede hacer muchas reservas.
- CLASES (1) ↔ (N) RESERVAS — Una clase puede tener muchas reservas.
- EMPLEADOS (1) ↔ (N) CLASES — Un instructor puede dar muchas clases.
- SALAS (1) ↔ (N) CLASES — Una sala puede acoger muchas clases.
- SOCIOS (1) ↔ (N) PAGOS — Un socio puede tener múltiples pagos.

3. Creación de la base de datos (SQL)

Script completo de creación de la base de datos para MySQL/MariaDB. Se incluyen todas las tablas con sus claves primarias, foráneas, tipos de datos y restricciones básicas.

```
-- =====
-- FitLife Gym - Script de Creación de Base de Datos
-- Módulo 0372 - Gestión de Bases de Datos
-- 1º ASIR - Proyecto Intermodular
-- =====
```

```
CREATE DATABASE IF NOT EXISTS fitlife_gym
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE fitlife_gym;

-- Tabla: membresias
CREATE TABLE membresias (
  id_membresia INT AUTO_INCREMENT PRIMARY KEY,
  tipo VARCHAR(50) NOT NULL,
  precio_mensual DECIMAL(8,2) NOT NULL,
  descripcion TEXT
);

-- Tabla: socios
CREATE TABLE socios (
  id_socio INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(150) NOT NULL,
  dni VARCHAR(9) UNIQUE NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  telefono VARCHAR(15),
  fecha_alta DATE NOT NULL DEFAULT (CURDATE()),
  id_membresia INT NOT NULL,
  FOREIGN KEY (id_membresia) REFERENCES membresias(id_membresia)
);

-- Tabla: empleados
CREATE TABLE empleados (
  id_empleado INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellidos VARCHAR(150) NOT NULL,
  puesto VARCHAR(80) NOT NULL,
  especialidad VARCHAR(100),
  email VARCHAR(150) UNIQUE,
  telefono VARCHAR(15),
  salario DECIMAL(10,2)
);

-- Tabla: salas
CREATE TABLE salas (
  id_sala INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  capacidad INT NOT NULL,
  equipamiento TEXT
);

-- Tabla: clases
CREATE TABLE clases (
  id_clase INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion TEXT,
  capacidad_max INT NOT NULL,
  horario_inicio TIME NOT NULL,
  horario_fin TIME NOT NULL,
```

```

dia_semana    ENUM('Lunes','Martes','Miercoles','Jueves',
                  'Viernes','Sabado','Domingo') NOT NULL,
id_sala       INT          NOT NULL,
id_empleado   INT          NOT NULL,
FOREIGN KEY (id_sala)      REFERENCES salas(id_sala),
FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)
);

-- Tabla: reservas
CREATE TABLE reservas (
  id_reserva    INT          AUTO_INCREMENT PRIMARY KEY,
  id_socio      INT          NOT NULL,
  id_clase      INT          NOT NULL,
  fecha_reserva DATE        NOT NULL DEFAULT (CURDATE()),
  estado        ENUM('confirmada','cancelada','pendiente')
                NOT NULL DEFAULT 'confirmada',
  FOREIGN KEY (id_socio) REFERENCES socios(id_socio),
  FOREIGN KEY (id_clase) REFERENCES clases(id_clase),
  UNIQUE KEY uq_reserva (id_socio, id_clase, fecha_reserva)
);

-- Tabla: pagos
CREATE TABLE pagos (
  id_pago       INT          AUTO_INCREMENT PRIMARY KEY,
  id_socio      INT          NOT NULL,
  importe       DECIMAL(8,2) NOT NULL,
  fecha_pago    DATE        NOT NULL DEFAULT (CURDATE()),
  metodo_pago   ENUM('efectivo','tarjeta','transferencia','domiciliacion')
                NOT NULL,
  concepto      VARCHAR(200),
  FOREIGN KEY (id_socio) REFERENCES socios(id_socio)
);

```

4. Inserción y gestión de datos

Se insertan datos de ejemplo coherentes con la operativa real de un gimnasio, suficientes para probar todas las consultas.

```

-- =====
--  FitLife Gym - Script de Inserción de Datos de Ejemplo
--  =====

USE fitlife_gym;

-- Membresías
INSERT INTO membresias (tipo, precio_mensual, descripcion) VALUES
  ('Básica', 29.90, 'Acceso sala de musculación L-V'),
  ('Premium', 49.90, 'Acceso ilimitado + clases grupales'),
  ('Anual', 299.00, 'Pago único anual, acceso completo');

-- Empleados

```

```
INSERT INTO empleados (nombre, apellidos, puesto, especialidad, email, salario)
VALUES
    ('Carlos', 'García Ruiz', 'Instructor', 'Yoga y Pilates',
    'carlos@fitlife.es', 1800.00),
    ('Laura', 'Martínez López', 'Instructor', 'CrossFit y HIIT',
    'laura@fitlife.es', 1900.00),
    ('Marcos', 'Sánchez Vega', 'Recepción', NULL,
    'marcos@fitlife.es', 1400.00),
    ('Sofía', 'Pérez Díaz', 'Instructor', 'Spinning y Cardio',
    'sofia@fitlife.es', 1850.00);

-- Salas
INSERT INTO salas (nombre, capacidad, equipamiento) VALUES
    ('Sala Cardio', 30, 'Cintas, bicicletas estáticas, elípticas'),
    ('Sala Grupal A', 20, 'Colchonetas, pesas ligeras, espejo'),
    ('Sala Spinning', 15, 'Bicicletas spinning, sistema de sonido'),
    ('Sala Musculación', 40, 'Maquinaria de fuerza, pesas libres, racks');

-- Socios
INSERT INTO socios (nombre, apellidos, dni, email, telefono, fecha_alta,
id_membresia) VALUES
    ('Ana', 'Fernández Gil', '12345678A', 'ana@email.com', '600111222',
    '2024-09-01', 2),
    ('Pedro', 'López Moreno', '23456789B', 'pedro@email.com', '600222333',
    '2024-10-15', 1),
    ('Lucía', 'Gómez Torres', '34567890C', 'lucia@email.com', '600333444',
    '2025-01-10', 3),
    ('Javier', 'Hernández Ruiz', '45678901D', 'javier@email.com', '600444555',
    '2025-02-20', 2),
    ('Marta', 'Jiménez Vidal', '56789012E', 'marta@email.com', '600555666',
    '2025-03-05', 1);

-- Clases
INSERT INTO clases (nombre, descripcion, capacidad_max, horario_inicio,
horario_fin, dia_semana, id_sala, id_empleado) VALUES
    ('Yoga Matutino', 'Clase de yoga para principiantes', 15, '09:00',
    '10:00', 'Lunes', 2, 1),
    ('HIIT Intensivo', 'Entrenamiento de alta intensidad', 12, '19:00',
    '20:00', 'Martes', 2, 2),
    ('Spinning', 'Ciclismo indoor con música', 15, '18:00',
    '19:00', 'Miercoles', 3, 4),
    ('Pilates', 'Fortalecimiento y flexibilidad', 15, '10:00',
    '11:00', 'Jueves', 2, 1),
    ('CrossFit', 'Entrenamiento funcional en equipo', 10, '07:00',
    '08:00', 'Viernes', 2, 2);

-- Reservas
INSERT INTO reservas (id_socio, id_clase, fecha_reserva, estado) VALUES
    (1, 1, '2025-05-05', 'confirmada'),
    (1, 3, '2025-05-07', 'confirmada'),
    (2, 2, '2025-05-06', 'confirmada'),
    (3, 1, '2025-05-05', 'confirmada'),
    (4, 5, '2025-05-09', 'confirmada'),
    (5, 2, '2025-05-06', 'cancelada'),
    (2, 4, '2025-05-08', 'confirmada');

-- Pagos
INSERT INTO pagos (id_socio, importe, fecha_pago, metodo_pago, concepto) VALUES
    (1, 49.90, '2025-05-01', 'domiciliacion', 'Cuota Mayo - Premium'),
```

```
(2, 29.90, '2025-05-01', 'domiciliacion', 'Cuota Mayo - Básica'),
(3, 299.00, '2025-01-10', 'tarjeta', 'Membresía Anual'),
(4, 49.90, '2025-05-01', 'tarjeta', 'Cuota Mayo - Premium'),
(5, 29.90, '2025-05-02', 'efectivo', 'Cuota Mayo - Básica');
```

5. Consultas y administración básica

5.1 Consultas útiles del sistema

Consulta 1 — Listado completo de socios con su membresía

```
SELECT s.id_socio, s.nombre, s.apellidos, s.dni, s.email,
       m.tipo AS membresia, m.precio_mensual
FROM socios s
JOIN membresias m ON s.id_membresia = m.id_membresia
ORDER BY s.apellidos, s.nombre;
```

Muestra todos los socios con el tipo de membresía que tienen contratada. Útil para la recepción.

Consulta 2 — Reservas de un socio concreto con datos de clase

```
SELECT r.id_reserva, r.fecha_reserva, r.estado,
       c.nombre AS clase, c.dia_semana, c.horario_inicio,
       e.nombre AS instructor
FROM reservas r
JOIN clases c ON r.id_clase = c.id_clase
JOIN empleados e ON c.id_empleado = e.id_empleado
WHERE r.id_socio = 1
ORDER BY r.fecha_reserva;
```

Permite ver el historial de reservas de un socio específico, incluyendo instructor asignado.

Consulta 3 — Ocupación actual de cada clase

```
SELECT c.nombre AS clase, c.dia_semana, c.horario_inicio,
       c.capacidad_max,
       COUNT(r.id_reserva) AS plazas_ocupadas,
       (c.capacidad_max - COUNT(r.id_reserva)) AS plazas_libres
FROM clases c
LEFT JOIN reservas r ON c.id_clase = r.id_clase
                    AND r.estado = 'confirmada'
GROUP BY c.id_clase
ORDER BY plazas_libres ASC;
```

Muestra cuántas plazas quedan disponibles en cada clase. Imprescindible para gestionar el aforo.

Consulta 4 — Ingresos del mes actual por método de pago

```
SELECT metodo_pago,
       COUNT(*) AS num_pagos,
```

```
SUM(importe) AS total_euros
FROM pagos
WHERE MONTH(fecha_pago) = MONTH(CURDATE())
AND YEAR(fecha_pago) = YEAR(CURDATE())
GROUP BY metodo_pago
ORDER BY total_euros DESC;
```

Resumen financiero mensual agrupado por forma de pago. Útil para la gestión contable.

Consulta 5 — Socios con membresía Premium que tienen reservas esta semana

```
SELECT DISTINCT s.nombre, s.apellidos, s.email, m.tipo
FROM socios s
JOIN membresias m ON s.id_membresia = m.id_membresia
JOIN reservas r ON s.id_socio = r.id_socio
WHERE m.tipo = 'Premium'
AND r.estado = 'confirmada'
AND WEEK(r.fecha_reserva) = WEEK(CURDATE())
ORDER BY s.apellidos;
```

Identifica socios Premium activos durante la semana. Permite acciones de fidelización.

5.2 Gestión de usuarios y permisos

En un entorno real, la base de datos de FitLife tendría al menos tres niveles de acceso:

```
-- Usuario para la aplicación web (solo lectura e inserción básica)
CREATE USER 'fitlife_app'@'localhost' IDENTIFIED BY 'App_S3cure!';
GRANT SELECT, INSERT, UPDATE ON fitlife_gym.socios TO
'fitlife_app'@'localhost';
GRANT SELECT, INSERT ON fitlife_gym.reservas TO
'fitlife_app'@'localhost';
GRANT SELECT ON fitlife_gym.clases TO
'fitlife_app'@'localhost';

-- Usuario para el administrador del sistema
CREATE USER 'fitlife_admin'@'localhost' IDENTIFIED BY 'Admin_S3cure!';
GRANT ALL PRIVILEGES ON fitlife_gym.* TO 'fitlife_admin'@'localhost';

-- Usuario de solo lectura para informes
CREATE USER 'fitlife_reportes'@'localhost' IDENTIFIED BY 'Report_S3cure!';
GRANT SELECT ON fitlife_gym.* TO 'fitlife_reportes'@'localhost';

FLUSH PRIVILEGES;
```

5.3 Copias de seguridad y exportación

La administración de la base de datos incluye las siguientes tareas rutinarias:

Copia de seguridad completa (mysqldump)


```
# Backup completo de la base de datos
mysqldump -u fitlife_admin -p fitlife_gym > backup_fitlife_$(date +%Y%m%d).sql

# Backup solo de la estructura (sin datos)
mysqldump -u fitlife_admin -p --no-data fitlife_gym > estructura_fitlife.sql

# Restaurar desde backup
mysql -u fitlife_admin -p fitlife_gym < backup_fitlife_20250503.sql
```

Exportación de datos para informes

```
-- Exportar socios a CSV
SELECT s.nombre, s.apellidos, s.email, m.tipo
INTO OUTFILE '/tmp/socios_fitlife.csv'
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM socios s JOIN membresias m ON s.id_membresia = m.id_membresia;
```

Se recomienda automatizar las copias de seguridad con un cron job diario (Linux) o con el Programador de tareas (Windows Server), guardando los archivos en una unidad de red separada con retención de 30 días.